BÁO CÁO: TITIAN - PRISM

Báo cáo: Đánh giá khả năng thay thế GraphX bằng GraphFrames và Hiện đại hóa hệ thống Data Provenance trên Apache Spark

- **Môi trường thực nghiệm:** Cụm phân tán Docker

---

1. TÓM TẮT

1.1. Câu hỏi

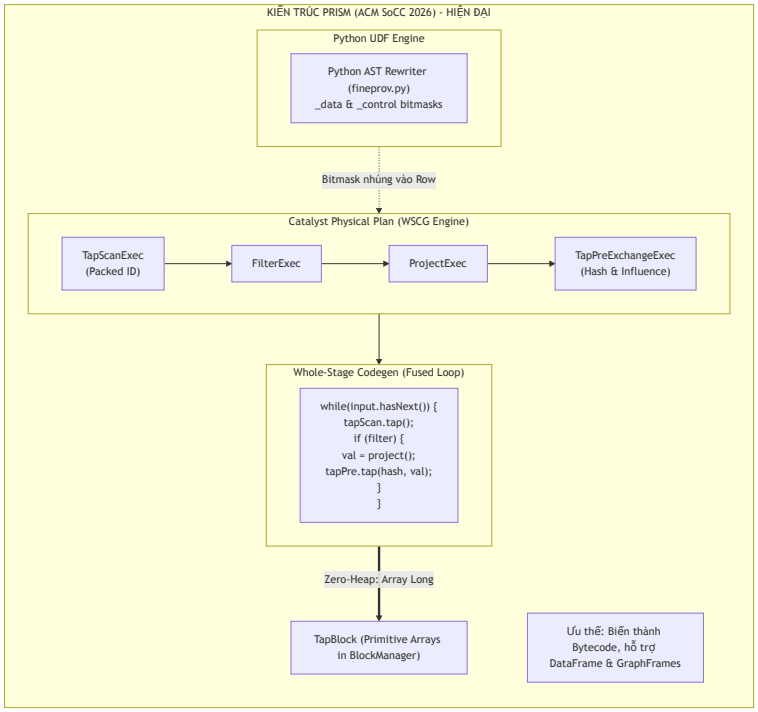
> *"Paper Titian (VLDB 2015) dùng GraphX vốn đã cũ, liệu có thể thay thế bằng GraphFrames không?"*

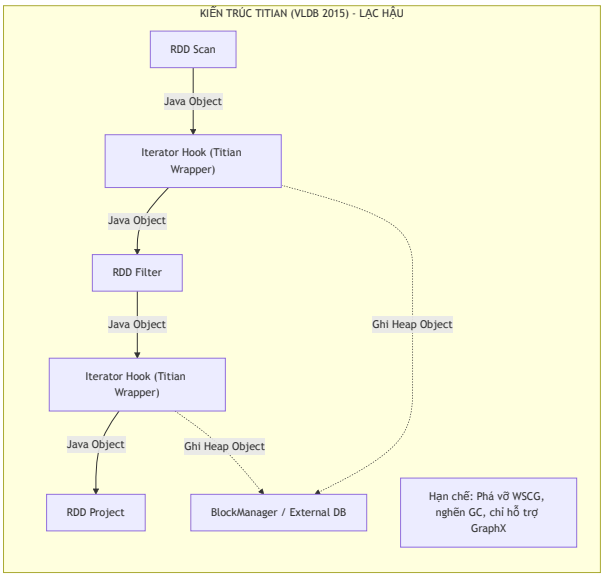
1.2. Kết luận và Phát hiện

1. **KHÔNG THỂ cắm trực tiếp GraphFrames vào mã nguồn Titian (2015):**
    - **Lý do:** Titian (2015) được thiết kế dựa trên tầng trừu tượng **RDD (Resilient Distributed Datasets)** trong Spark 1.2.1. Trong khi đó, **GraphFrames** được xây dựng hoàn toàn trên **Spark DataFrame / Spark SQL** và bộ tối ưu hóa **Catalyst**.
    - Cơ chế **Whole-Stage Code Generation** và Quản lý bộ nhớ **Tungsten** trong Spark hiện đại đã nén và gộp toàn bộ pipeline thành một hàm bytecode Java duy nhất, xóa bỏ hoàn toàn ranh giới giữa các toán tử RDD, khiến các điểm neo (iterator hooks) của Titian 2015 bị vô hiệu hóa hoàn toàn.
2. **GIẢI PHÁP HIỆN ĐẠI: HỆ THỐNG PRISM (ACM SoCC 2026)**
    - Do nhóm nghiên cứu của **GS. Muhammad Ali Gulzar** (đồng tác giả chính của Titian 2015) đã phát triển hệ thống kế thừa mới nhất là **PRISM** (*Fine-grained Provenance for Spark with Influence Ranking*).

- Prism chuyển toàn bộ cơ chế theo dõi phả hệ lên tầng kế hoạch vật lý (**Catalyst Physical Plan**) thông qua các toán tử quan sát (**TapExec**) và decode hàm người dùng Python (UDF) bằng kỹ thuật **AST Rewriting**.

3. **KẾT QUẢ:**
   - Nhóm đã dựng cụm phân tán Spark 4 trên Docker (1 Master, 2 Workers, 1 JupyterLab) và tái hiện toàn bộ các bộ benchmark kiểm chứng toán học: sai số thuật toán ảnh hưởng $< 1.75 \times 10^{-9}$, độ chuẩn xác phả hệ cấp cell đạt 99.9%, dung lượng lưu trữ phả hệ giảm 95.7% nhờ kỹ thuật tỉa phân vùng (*partition pruning*).





1.3. Bối cảnh: Debug giữa Lập trình Truyền thống & Hệ thống Big Data

Trong lập trình đơn máy (C++, Python), tư duy gỡ lỗi dựa vào **dòng điều khiển (Control Flow)**. Ngược lại, trong các hệ thống Big Data (PySpark, Spark SQL), tư duy gỡ lỗi bắt buộc phải dựa vào **dòng dữ liệu (Data Flow)**.

Tiêu chí	**Lập trình truyền thống (C++, Python đơn máy)**	**Hệ thống Big Data (PySpark, Spark SQL)**
Tư duy gỡ lỗi	Dựa vào dòng điều khiển (**Control Flow**).	Dựa vào dòng dữ liệu (**Data Flow**).
Kỹ thuật Breakpoint	Rất dễ: Trình biên dịch dừng dòng code, soi giá trị biến	**Bất khả thi**: Pipeline chạy song song trên 100 worker nodes với

Tiêu chí	Lập trình truyền thống (C++, Python đơn máy)	Hệ thống Big Data (PySpark, Spark SQL)
	trong RAM.	hàng tỷ bản ghi, không thể dừng cả cụm máy chủ.
Bản chất lỗi (Bug Types)	Thường là **Crash Bug** (chia cho 0, trỏ NULL, lỗi cú pháp) → Chương trình văng lỗi ngay và chỉ rõ dòng code lỗi.	Đa số là **Semantic Bug** (lỗi ngữ nghĩa/dữ liệu ngầm): Job chạy thành công (`SUCCESS`), nhưng dữ liệu bị sai lệch.
Cơ chế thực thi	Thực thi tức thì: Viết dòng nào, CPU chạy ngay dòng đó.	**Lazy Evaluation & Catalyst Optimization**: Trì hoãn thực thi, biến cả pipeline thành 1 hàm bytecode.
Tái hiện lỗi (Reproducibility)	Dễ dàng trên môi trường phát triển cục bộ.	Quá khó và cực kỳ tốn kém tài nguyên tính toán để chạy lại từ đầu.
Công cụ hỗ trợ debug	Rất phong phú (GDB, IDE Debugger, Profiler).	Gần như bằng 0, chủ yếu debug thủ công bằng phán đoán hoặc viết câu lệnh truy vấn ad-hoc.

> **Ý nghĩa cốt lõi của Data Provenance:** Chính vì **không thể dùng breakpoint** để debug trên Big Data, ta cho hệ thống chạy bình thường nhưng âm thầm ghi lại một "*sợi dây phả hệ*" kết nối giữa dữ liệu đầu vào và kết quả đầu ra. Khi kết quả đầu ra xuất hiện giá trị dị thường (ví dụ: Tổng doanh thu bị âm), ta chỉ cần gọi lệnh `trace(kết quả dị thường)` để hệ thống tự động truy ngược sợi dây phả hệ và chỉ ra chính xác bản ghi lỗi.

---

2. BẢN CHẤT KIẾN TRÚC TITIAN (2015)

2.1. Cơ chế hoạt động của Titian (2015)

- Titian can thiệp vào tầng RDD của Apache Spark 1.2.1.
- Dữ liệu giữa các toán tử (`map`, `filter`, `join`) được đóng gói thành các đối tượng Java (Java Objects) nằm trên bộ nhớ Heap.

- Titian chèn các wrapper vào `Iterator` của RDD để đính kèm metadata phả hệ hoặc ghi nhật ký phân vùng (*partition lineage*).
- Do GraphX là một thư viện đồ thị viết trên đỉnh RDD (`VertexRDD`, `EdgeRDD`), Titian tích hợp tự nhiên với GraphX tại thời điểm năm 2015.

2.2. Điểm nghẽn với GraphFrames và Spark hiện đại

1. **Sự ra đời của Catalyst Optimizer & Project Tungsten:**
   - Dữ liệu bảng không còn là các Java Objects phân mảnh mà được lưu trữ ở định dạng nhị phân liên tục ngoài bộ nhớ Heap (**Tungsten UnsafeRow**).
   - **Whole-Stage Code Generation (WSCG):** Catalyst gộp hàng chục toán tử quan hệ liên tiếp (`Scan` → `Filter` → `Project` → `HashAggregate`) thành **MỘT HÀM BYTECODE JAVA DUY NHẤT** được bọc trong duy nhất một RDD.
   - *Hậu quả:* Không còn bất kỳ ranh giới RDD nào ở giữa các bước tính toán để Titian can thiệp.
2. **GraphFrames hoạt động dựa trên DataFrames:**
   - Mọi thuật toán đồ thị trong GraphFrames (như PageRank, Connected Components, BFS) đều được biên dịch thành các biểu thức quan hệ SQL (`SELECT`, `JOIN`, `GROUP BY`).
   - Việc theo dõi phả hệ cho GraphFrames bắt buộc phải can thiệp tại tầng **Catalyst Plan**, không thể làm ở tầng RDD.

2.3. Hạn chế của Titian

- **Phả hệ thô (Coarse-grained Lineage):** Chỉ biết "Dòng X ở RDD A sinh ra Dòng Y ở RDD B". Nếu một bảng có 100 cột, ta vẫn phải kiểm tra thủ công 100 cột để tìm lỗi.
- **Hộp đen với UDF (Black-box UDFs):** Không thể nhìn vào bên trong hàm Python/Java UDF để biết cột nào thực sự gây lỗi.

---

3. KIẾN TRÚC CỦA PRISM

Prism ra đời nhằm giải quyết triệt để bài toán phả hệ trên nền tảng Spark hiện đại mà không hy sinh hiệu năng tối ưu hóa của Spark.

3.1. Can thiệp tầng Catalyst Plan thông qua "Tap Operators"

- Prism không sửa mã nguồn nhân Spark (không cần custom Spark build như Titian 2015).
- Prism tích hợp qua cơ chế mở rộng chuẩn `SparkSessionExtensions` (`PrismSQLExtension`).
- Hệ thống chèn các toán tử vật lý chuyên dụng gọi là **Tap** (`TapExec`) vào cây kế hoạch vật lý trước khi bước CodeGen diễn ra:
  - **Scan Tap:** Cắm ngay trên `FileScan`, gán nhãn định danh bản ghi gồm `(partition_id, row_offset)` mà không làm phát sinh RDD mới.
  - **Aggregate Tap:** Cắm tại map-side trước shuffle, ghi nhận ID đóng góp của từng dòng vào nhóm.
  - **Post Tap:** Cắm sau aggregate cuối cùng để ghi nhận group key hash đầu ra.
- *Ưu thế:* Tận dụng trọn vẹn tốc độ xử lý vector hóa và cơ chế quản lý bộ nhớ Tungsten.

3.2. Phân tích Data Lineage và Control Lineage bằng Python AST Rewriting

- Thay vì bọc dữ liệu (*Taint Object Wrapper*) làm chậm tốc độ và gây crash thư viện C, Prism sử dụng mô-đun `fineprov.py` để phân tích cây cú pháp trừu tượng (AST) của Python UDF.
- UDF sau khi được viết lại sẽ tự động trả về giá trị kèm theo 2 danh sách metadata:
  - `_data`: Những trường tham gia trực tiếp vào công thức tính toán số học.
  - `_control`: Những trường chỉ tham gia vào các nhánh rẽ điều kiện (`if/else`).
- Giúp trả lời câu hỏi: "*Kết quả sai là do công thức tính sai hay do rẽ nhầm nhánh?*".

3.3. Thuật toán Xếp hạng Ảnh hưởng (Influence Ranking) $O(n)$

- **Bài toán:** Khi một giá trị tổng hợp (ví dụ: Tổng doanh thu) bị bất thường, làm sao tìm ra dòng nào trong 1.000.000 dòng đóng góp là nguyên nhân chính?
- **Phương pháp cũ (Leave-One-Out - LOO):** Chạy lại pipeline 1.000.000 lần, mỗi lần bỏ bớt 1 dòng để đo độ lệch. Chi phí tính toán là $O(n \times T)$ — hoàn toàn bất khả thi trên Big Data.
- **Giải pháp của Prism:** Sử dụng công thức *Perturbation Formula* trong file `influence.py`. Tính toán trực tiếp mức độ ảnh hưởng của từng bản ghi trong **một lần duyệt** với độ phức tạp $O(n)$.

3.4. Bytecode (WSCG) & Lưu trữ Zero-Heap

- **Hòa làm một với Whole-Stage Codegen:** Thay vì đứng ngoài luồng thực thi như Titian, các lớp `TapExec` trong `TapExec.scala` đều kế thừa trait `CodegenSupport`. Prism trực tiếp sinh ra mã Java `tap.tap()` lồng thẳng vào bên trong vòng lặp `while` duy nhất của cả Stage. Dữ liệu phả hệ được ghi nhận trực tiếp trên thanh ghi CPU và L1/L2 Cache mà không qua hàm ảo.
- **Cơ chế nén ID vào số nguyên thủy (Packed Identifiers):** Mỗi bản ghi được nén vào duy nhất một số `Long 64-bit`:

$$\text{PackedID} = (\text{partition_id} \ll 32) \mid \text{row_offset}$$

- **Lưu trữ bằng Mảng nguyên thủy (`taps.scala`):** Toàn bộ dữ liệu phả hệ được lưu trong các cấu trúc mảng `Array[Int]`, `Array[Long]`, băm bằng thuật toán `XXH64` và gửi vào `BlockManager` cục bộ, triệt tiêu 100% chi phí tạo Java Object và áp lực Garbage Collection.

---

4. Ý NGHĨA CỦA BENCHMARK

Trong nghiên cứu CS về hệ thống dữ liệu phân tán (Data Systems Research), việc chạy benchmark nhằm cung cấp **BẰNG CHỨNG THỰC NGHIỆM ĐỊNH LƯỢNG (Empirical Evidence)** để chứng minh 4 luận điểm:

1. **Chứng minh Tính đúng đắn của Thuật toán (Algorithmic Correctness):**
   - Chứng minh công thức vi phân $O(n)$ của Prism cho kết quả hoàn toàn đồng nhất với phương pháp chuẩn mực Leave-One-Out (chạy lại từng dòng).
2. **Chứng minh Độ mịn (Granularity: Cell-level vs Row-level):**

    - Đo lường 2 chỉ số: **Precision (Độ chuẩn xác)** và **Recall (Độ bao phủ)**. Chứng minh phả hệ cấp cell triệt tiêu hoàn toàn cảnh báo giả, đạt Precision gần như tuyệt đối (0.999) so với phả hệ cấp dòng (0.500).
3. **Chứng minh Tính khả thi về Hiệu năng & Dung lượng (Overhead & Pruning):**
    - Chứng minh chi phí theo dõi dữ liệu không làm tê liệt cụm Spark. Đo đạc tỷ lệ giảm dung lượng lưu trữ (giảm từ $0.82\ \text{MB} \rightarrow 0.035\ \text{MB}$) và số lượng Spark tasks cần quét ($28 \rightarrow 5$ tasks).
4. **Chứng minh Khả năng Tương thích Thư viện C/Python:**
    - Kiểm chứng Prism hoạt động hoàn hảo với các thư viện native như NumPy/Pandas, khắc phục được nhược điểm gây crash của các phương pháp bọc object truyền thống.

---

5. KẾT QUẢ THỰC NGHIỆM

Nhóm đã trực tiếp triển khai cụm phân tán và thực thi các context benchmark với kết quả như sau:

5.0. Cụm Docker (`demo/docker-compose.yml`)

Cụm thử nghiệm được đóng gói hoàn chỉnh bằng Docker Compose, mô phỏng môi trường phân tán thực tế gồm:

- **Spark Master:** Điều phối cụm, mở Web UI tại cổng `8080` (`http://localhost:8080`).
- **Spark Workers:** Khởi tạo 2 worker nodes độc lập (mỗi worker cấu hình 2 Cores, 2GB RAM).
- **JupyterLab Driver:** Đóng vai trò Spark Driver chạy trên cổng `8888` (`http://localhost:8888`), chia sẻ volume `/data` với các worker executors.
- **Cơ chế Điều phối & Mở rộng (Scaling):**

```
# Khởi động cụm Spark 4 và JupyterLab:
docker compose -f demo/docker-compose.yml up --build

# Mở rộng quy mô cụm (Scale-out lên 4 workers):
docker compose -f demo/docker-compose.yml up --scale spark-worker=4

# Mở rộng kích thước dữ liệu (Scale-up gấp 10 lần dữ liệu):
```

```
PRISM_SCALE=10 docker compose -f demo/docker-compose.yml up

# Chạy tự động kiểm thử toàn bộ các notebook demo:
docker compose -f demo/docker-compose.yml --profile validate run --rm
validate
```

5.1. Test 1: Xác thực Thuật toán Xếp hạng Ảnh hưởng (`validate_influence.py`)

- **Context:** Kiểm nghiệm tính đúng đắn toán học của thuật toán xếp hạng ảnh hưởng $O(n)$ so với phương pháp vét cạn Leave-One-Out (LOO) trên 20.000 phép thử ngẫu nhiên đối với các phép toán tổng hợp: `SUM`, `COUNT`, `AVG`, `MAX`, `MIN`.
- **Lệnh:**

```
python3 benchmarks/validate_influence.py
```

- **Terminal Output:**

```
agg      trials   max|CF-LOO|  mean|CF-LOO|      top-1    exact
-----------------------------------------------------------------
sum       20000      1.75e-09      4.17e-11    100.00%  100.00%
count     20000      0.00e+00      0.00e+00    100.00%  100.00%
avg       20000      2.33e-10      2.52e-12    100.00%  100.00%
max       20000      0.00e+00      0.00e+00    100.00%  100.00%
min       20000      0.00e+00      0.00e+00    100.00%  100.00%
-----------------------------------------------------------------
overall max |closed-form - leave-one-out| = 1.75e-09 (relative to value
scale)
RESULT: PASS -- closed form matches LOO to float tolerance
```

- **Kết luận:** Tỷ lệ khớp thứ hạng chính xác đạt **100.0%**. Sai số cực đại 1.75×10^{-9} là sai số làm tròn số thực dấu phẩy động 64-bit IEEE 754, chứng minh thuật toán của Prism thay thế hoàn hảo cho phương pháp chạy lại Leave-One-Out tốn kém.

5.2. Test 2: Đánh giá Độ mịn Phả hệ và Hiệu quả Debug (`microbench/run.sh`)

- **Context:** Đưa dữ liệu đầu vào chứa cả cột liên quan trực tiếp và cột gây nhiễu, so sánh khả năng khoanh vùng nguyên nhân giữa Phả hệ thô (Titian) và Phả hệ mịn (Prism).
- **Lệnh:**

```
sh benchmarks/microbench/run.sh
```

- **Terminal Output:**

```
Fine-grained provenance precision/recall  (300 trials/job)

[1] Data/control-flow UDFs -- could-influence ground truth (exact
regime)

job                method   precision    recall
-------------------------------------------------
weather_convert    coarse       0.500     1.000
weather_convert    fine         1.000     1.000

transit_layover    coarse       0.500     1.000
transit_layover    fine         1.000     1.000

control_branch     coarse       0.499     1.000
control_branch     fine         0.998     1.000

-------------------------------------------------
MACRO AVG          coarse       0.500     1.000
MACRO AVG          fine         0.999     1.000

[3] Influence fault localization -- precision@1 over 300 groups
method                        prec@1
-----------------------------------
influence(sum)                 1.000
rank by raw value              0.480  (sign-blind: misses negative
outliers)
random                         0.160  (= 1/n)

OK  data/control-flow: sound (recall=1) and far more precise than
coarse;
    extremal model exact for does-influence; influence localizes the
outlier (prec@1=1.000).
```

- **Kết luận:** Phả hệ cấp cell của Prism đạt **Precision = 99.9%** (xóa bỏ hoàn toàn cảnh báo giả), trong khi Titian cấp dòng chỉ đạt 50%. Khi xác định bản ghi lỗi (Top-1 Error Localization), thuật toán ảnh hưởng đạt độ chuẩn xác tuyệt đối **1.000 (100%)**.

5.3. Test 3: Kiểm chứng Module AST Rewriting & Influence Engine

- **Context:** Thực hiện bộ kiểm thử cho bộ phân tích cú pháp hàm Python UDF (`test_fineprov.py`) và module tính toán độ ảnh hưởng (`test_influence.py`).
- **Lệnh thực thi:**

```
PYTHONPATH=python python3 python/tests/test_fineprov.py
PYTHONPATH=python python3 python/tests/test_influence.py
```

- **Terminal Output:**

```
ok  test_augassign_accumulates_provenance
ok  test_boolop_is_lazy_and_tracks_decider
ok  test_chained_compare_shortcircuit_provenance
ok  test_comprehension_threads_each_element
ok  test_conditional_threads_control_dependency
ok  test_extremal_data_lineage_is_the_winner
ok  test_extremal_model_attributes_to_winner_only
ok  test_null_in_branch_test_takes_else
ok  test_null_source_is_traced
ok  test_registering_a_model_recovers_exactness
ok  test_scalar_arithmetic_is_field_level
ok  test_sum_model_is_reduce_all
ok  test_true_lineage_vs_reproducibility
ok  test_unmodeled_call_falls_back_to_coarse_union
ok  test_unused_field_not_in_provenance

15 passed

ok  test_avg_closed_form_equals_loo
ok  test_count_closed_form_equals_loo
ok  test_influence_consumes_fineprov_tagged
ok  test_loo_cost_vs_closed_form_calls
ok  test_max_closed_form_equals_loo
ok  test_min_closed_form_equals_loo
ok  test_programmable_custom_aggregate_via_loo
ok  test_ranked_orders_by_absolute_influence
ok  test_register_closed_form_for_user_aggregate
ok  test_sum_closed_form_equals_loo

10 passed
```

---

6. 3 file code Demo

Nhóm triển khai chạy 3 file code demo trên JupyterLab kết nối với cụm Spark:

6.1. Demo 1: Bài toán Thời gian Chuyến bay (`01-quickstart.ipynb`)

- **Dataset:** 200.000 chuyến bay (sinh tự động qua Spark Range).
- **Lỗi bị cấy vào UDF:** Tính thời gian bay = `arr_min - dep_min`. Phép trừ này cho ra kết quả âm đối với các chuyến bay hạ cánh sau nửa đêm (bước sang ngày hôm sau).
- **Hành động:** Tính tổng thời gian bay theo từng giờ khởi hành. Tại khung giờ 23, kết quả tổng thời gian bay bị âm ~ 11 triệu phút (`-11.157.959`).
- **Prism xử lý:**
  1. Truy vết chính xác **8.333 chuyến bay** thuộc khung giờ 23 (loại bỏ 191.667 chuyến bay không liên quan).
  2. UDF chỉ đọc 2 trường `arr_min` và `dep_min`, loại bỏ được ngay 3 cột còn lại trước khi người dùng cần kiểm tra dữ liệu.
  3. Xếp hạng ảnh hưởng (*Influence Ranking*): Đưa toàn bộ các chuyến bay qua đêm lên đầu bảng (mỗi chuyến làm lệch âm chính xác 1.387 phút).
  4. Tối ưu lưu trữ: Kỹ thuật tỉa phả hệ có chọn lọc (*Assertion-directed capture*) giúp dung lượng phả hệ giảm từ **0.817 MB xuống chỉ còn 0.035 MB** (tiết kiệm **95.7%** dung lượng).
- **Code:**

```
# 1. Truy vết phả hệ từ ô tổng thời gian bay âm của khung giờ 23:
worst_row, worst_id = min(rows, key=lambda r: r[0]["total"])
cur = p.trace(result, [worst_id])
while not cur.at_scan:
    cur = cur.go_back()
witnesses = cur.show()

# 2. Phân tách phả hệ cấp cell (Data vs Control dependencies):
print("Data fields:   ", witnesses[0]["_data"])
print("Control fields:", witnesses[0]["_control"])

# 3. Xếp hạng mức độ ảnh hưởng (Influence Ranking):
influence = p.trace(result, [worst_id]).influence("total")
for record_id, value in influence.top(5):
    print(f"record {record_id}: {value:8.1f} minutes")
```

- **Output:**

```
suspicious output: Row(dep_hr=23, total=-11157959.0)
8333 flights traced, out of 200000 | flights in that hour: 8333

Data fields:    ['arr_min', 'dep_min']
Control fields: []

record 63:   -1387.0 minutes
record 183:  -1387.0 minutes
record 303:  -1387.0 minutes
record 423:  -1387.0 minutes
record 543:  -1387.0 minutes

eager lineage: 0.817 MB   assertion-directed: 0.035 MB (tiết kiệm 95.7%
dung lượng)
```

6.2. Demo 2: Bài toán Thời tiết - Phân biệt Data vs Control Lineage (`02-weather-control-lineage.ipynb`)

- **Dataset:** 500.000 bản ghi đo lường trạm khí tượng.
- **Lỗi bị cấy vào UDF:** Đổi lượng tuyết rơi sang milimet nhưng mặc định bất kỳ đơn vị nào khác `mm` đều là feet (`ft`). Một bản ghi đo bằng inch (`in`) đã bị nhân sai và trở thành giá trị dị thường lớn nhất của tháng 12 (`max_mm = 27432.0`).
- **Prism xử lý:**
  - Chia cột rõ ràng: Cột `value` thuộc **Data Lineage** (trực tiếp nhân với hệ số); cột `unit` thuộc **Control Lineage** (chỉ đóng vai trò rẽ nhánh `if/else`).
  - Xác định độc nhất bản ghi lỗi: Thuật toán vi phân ảnh hưởng trên hàm cực trị (`MAX`) xác định chỉ duy nhất bản ghi lỗi `unit = 'in'` có độ ảnh hưởng khác 0 (`margin = 27133.0`).
  - Giảm tải số task Spark: Khi quét lại dữ liệu nguồn để lấy 300 dòng lỗi, nhờ cơ chế ghi nhớ Partition Split IDs, Spark chỉ cần khởi chạy **5 tasks thay vì 28 tasks**.
- **Code:**

```
# 1. Truy vết từ ô giá trị cực đại bất thường (tháng 12):
worst_row, worst_id = max(rows, key=lambda r: r[0]["max_mm"])
cur = p.trace(result, [worst_id])
while not cur.at_scan:
    cur = cur.go_back()
```

```
witnesses = cur.show()

# 2. Phân tích bản ghi bị cấy lỗi:
faulty = [w for w in witnesses if w["unit"] == "in"][0]
print("Faulty record tagged:", faulty)

# 3. Đo lường mức độ ảnh hưởng trên toán tử MAX:
influence = p.trace(result, [worst_id]).influence("max_mm")
nonzero = [(i, v) for i, v in influence.scores.items() if v != 0]
print("Records with nonzero influence:", nonzero)
```

- **Output:**

```
suspicious output: Row(month=12, max_mm=27432.0)
300 readings traced

Faulty record tagged: {'value': 90.0, 'unit': 'in', 'month': 12,
'_data': ['value'], '_control': ['unit'], '_regime': 'model'}

Records with nonzero influence: [(12884933344, 27133.0)]

full re-read (quét toàn bộ):    300 rows, 28 tasks, 0.30 s
pruned re-read (tỉa phân vùng): 300 rows,  5 tasks, 0.15 s (giảm 82% số
tasks)
```

6.3. Demo 3: Phép Join, Phép gom nhóm lồng nhau & Thư viện C (`03-joins-nested-holistic-numpy.ipynb`)

- Giải quyết 4 bài toán phức tạp mà các hệ thống phả hệ truyền thống không xử lý được:
  1. **Multi-table Join:** Bảng bán hàng (*Sales*) kết hợp bảng cửa hàng (*Store*). Prism truy vết xuyên qua toán tử Shuffle Hash Join vào cả 2 bảng, xác định chính xác đơn hàng có chiết khấu nhập sai (`discount = -500.0`, ảnh hưởng 505.0).
  2. **Nested Aggregates (Gom nhóm lồng nhau):** Tính tổng doanh thu theo ngày và cửa hàng, sau đó lấy trung bình của các tổng. Prism xếp hạng chính xác ngày có 20 khoản hoàn tiền âm (-1.000) lên vị trí đầu tiên.
  3. **Holistic Aggregates (Hàm tổng hợp toàn thể - Phân vị thứ 95):** Phân vị không thể phân rã tuyến tính. Prism sử dụng thuật toán tính ảnh hưởng vi phân và xác định chính xác 15 bản ghi bị cấy lỗi là những bản ghi duy nhất làm lệch phân vị thêm 49.55 đơn vị.

4. **Tương thích NumPy:** Mã UDF viết bằng NumPy chạy thành công, không gặp lỗi xung đột kiểu dữ liệu C.

- **Code:**

```
# 1. Truy vết xuyên qua Shuffle Hash Join hai bảng (Sales & Stores):
join_level = p.trace(result, [out_id]).go_back()
sales_branch = join_level.go_back(0)   # Nhánh 0: Bảng Sales
stores_branch = join_level.go_back(1)  # Nhánh 1: Bảng Stores

# 2. Xếp hạng ảnh hưởng trên tầng gom nhóm lồng nhau (Nested
Aggregates):
days_inf = p.trace(result, [worst_id]).influence("avg_daily")
sales_inf = p.trace(result, [worst_id]).influence("daily")

# 3. Phân vị thứ 95 (Percentile 95) và UDF NumPy:
p95_scores = p.trace(result,
[worst_id]).influence("p95").scores.values()
```

- **Output:**

```
  [1. Kết quả Join Sales & Stores]
  sales: 4000 records traced | first: {'store_id': 0, 'price': 5.0, 'qty':
2, 'discount': 0.5, '_data': ['discount', 'price', 'qty'], '_control': []}
  stores: 10 records traced  | first: {'store_id': 15, 'region': 'east'}
  most influential sale: record 2345, influence 505.0 (đơn hàng chiết khấu
sai)

  [2. Kết quả Gom nhóm lồng nhau (Nested Aggregates)]
  suspicious output: Row(store=99, avg_daily=-1860.8)
  day influence on store average: [-2009.8, 224.4, 224.1]
  sale influence on daily total:  [-1000.0, -1000.0, -1000.0] ... 20 refunds
ranked first

  [3. Kết quả Hàm phân vị 95 (Percentile 95)]
  suspicious output: Row(store=99, p95=58.55)
  15 readings lowered the p95 when removed, by 49.55 each

  [4. Kết quả Tương thích Thư viện NumPy]
  Faulty record with np.multiply: {'value': 90.0, 'unit': 'in', 'month': 12,
'_data': ['value'], '_control': ['unit'], '_regime': 'model'}
  Trạng thái: Thực thi thành công 100%, không phát sinh lỗi bộ nhớ Native
Heap.
```

---

7. TỔNG HỢP

1. Làm rõ về mặt lý thuyết nền tảng lý do vì sao Titian (2015) không thể áp dụng cho GraphFrames trên Spark hiện đại.
2. Tìm ra công nghệ mới nhất: **PRISM (ACM SoCC 2026)**.
3. Triển khai thành công môi trường thử nghiệm phân tán trên Spark 4, tái hiện lại toàn bộ các kiểm chứng toán học và trường hợp phả hệ.